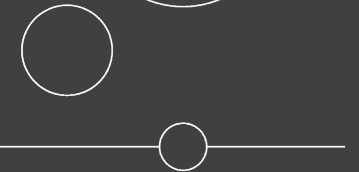


EVOP 2026

Introduction to Linux pt. 1

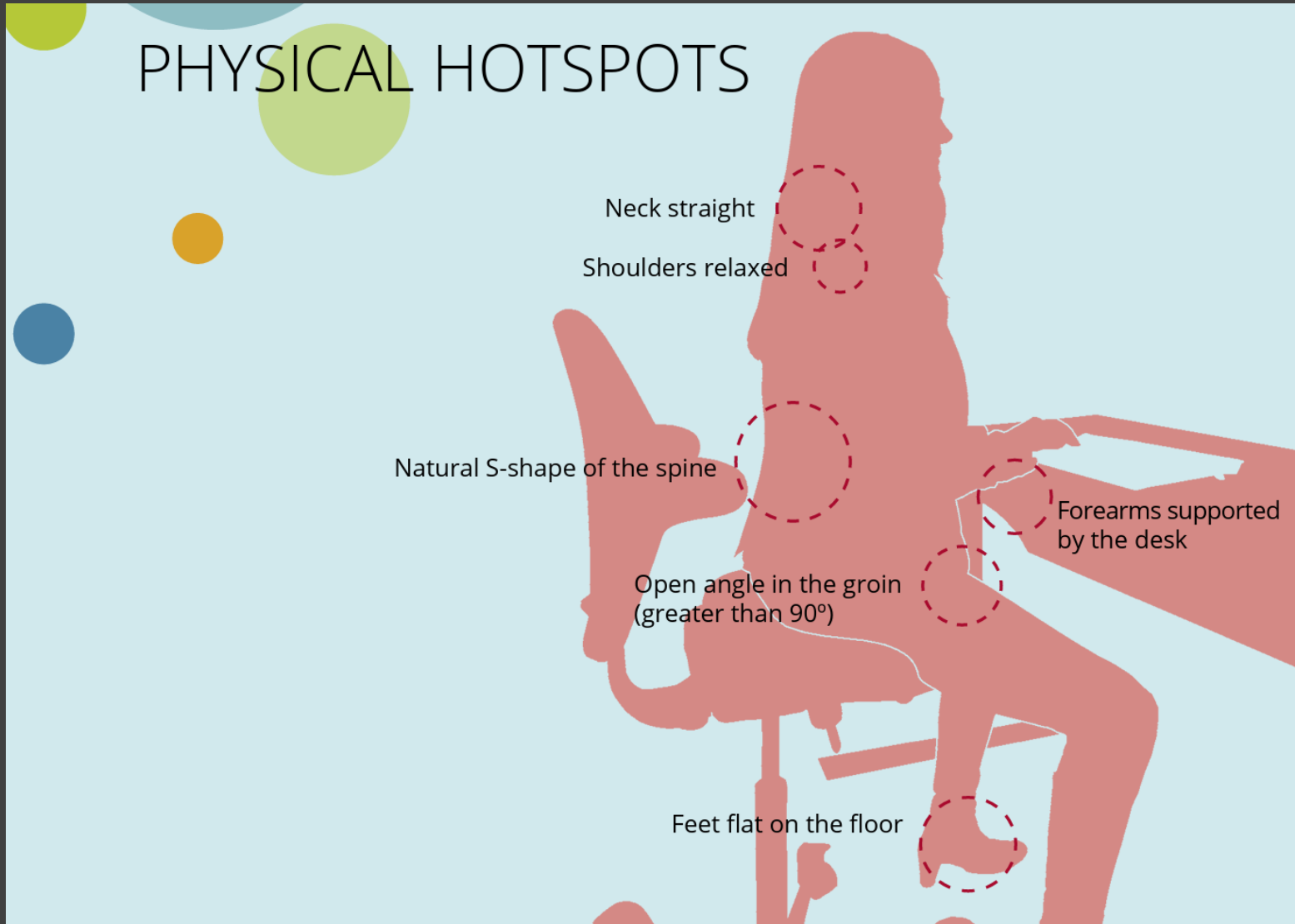
Dr. Isolde van Riemsdijk
Copenhagen University



KØBENHAVNS
UNIVERSITET

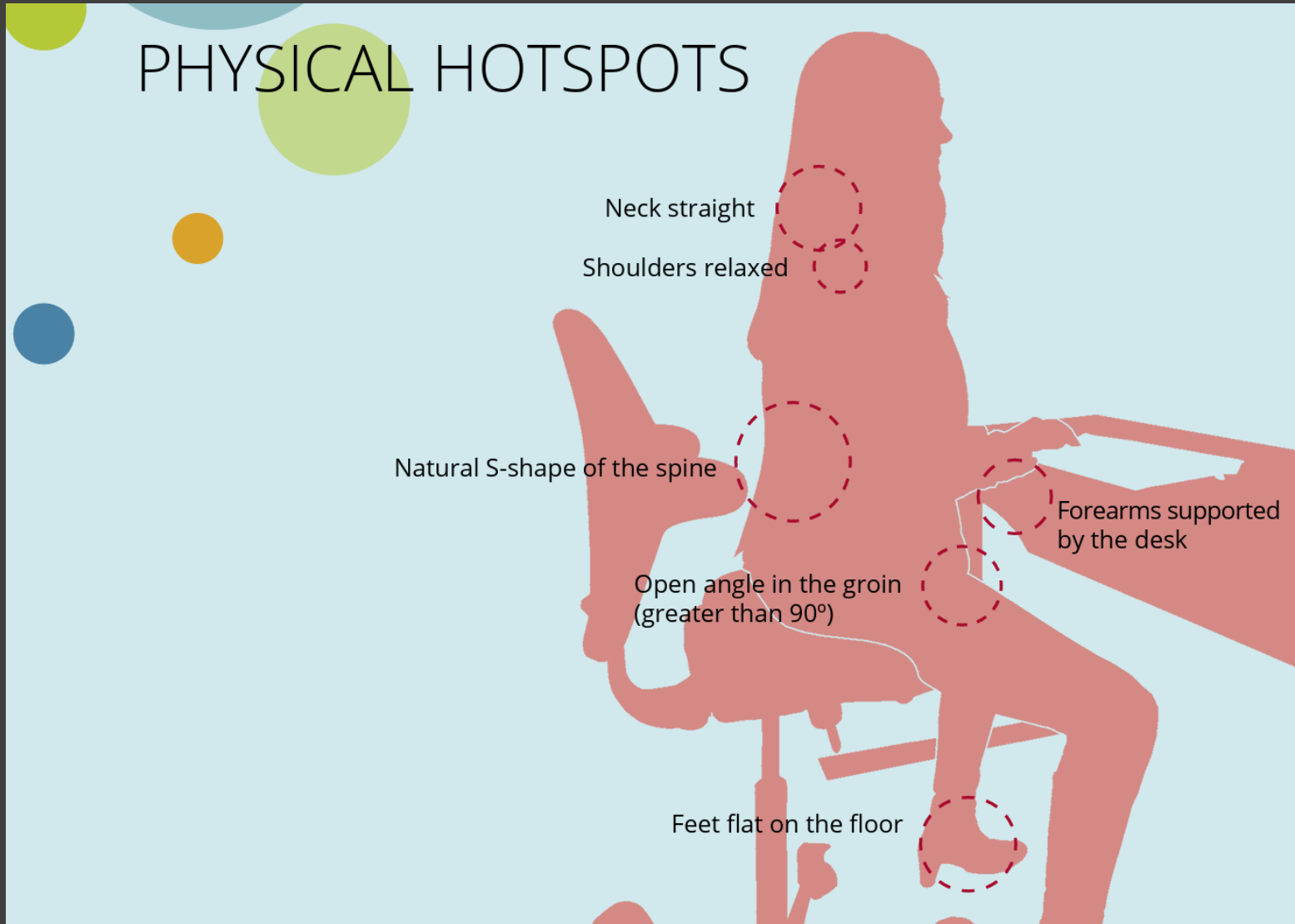
Welcome

Set up your work station



Welcome

Set up your work station



Welcome

Set up your work station

HOW TO ADJUST YOUR CHAIR

Try out all your chair's levers and adjustments

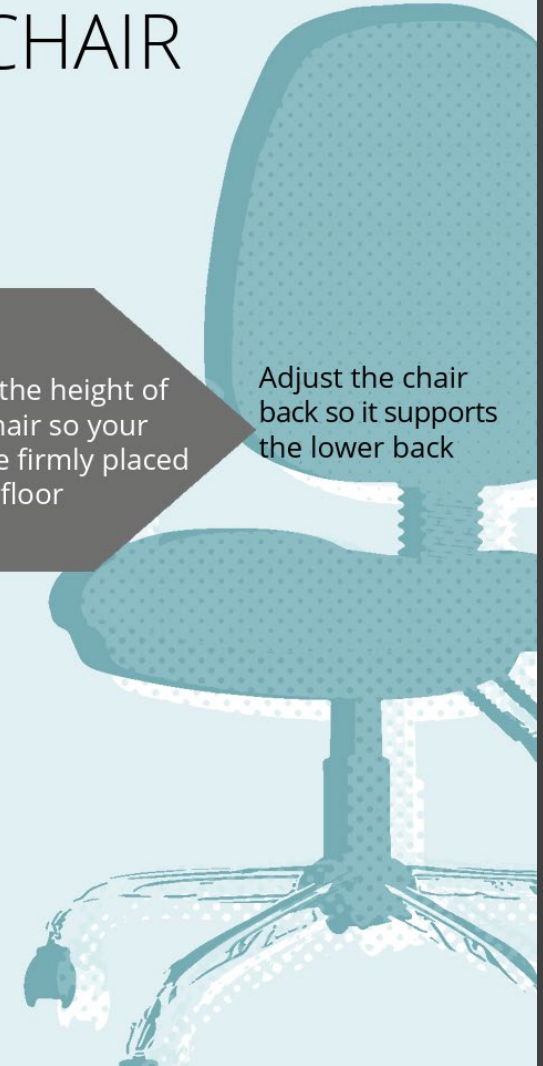


Sit well back in the chair so that your thighs are supported by the seat. Check that your clenched fist can fit between the back of your knee and the seat

If your seat can tilt, it can be good for your back

Adjust the height of your chair so your feet are firmly placed on the floor

Adjust the chair back so it supports the lower back



Welcome

And keep it tidy!

Logistics



Teaching Assistants

Second screen

Sticky notes

Set-up and log into the desktop

Slides are also on:

https://github.com/IsoldevR/Linux_Intro_EVOP_2026

Introduction of Unix & Terminal

What is Unix?

- Operating system
- Runs on any computer
- Designed for developing software
- Sharing of tools

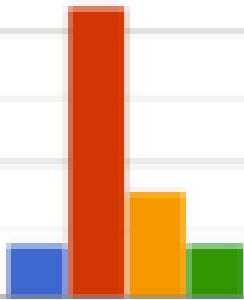
Free UNIX-like variants

- Ubuntu, Fedora, Debian

Introduction of Unix & Terminal

20

Not Used Basic Usage Competent Proficient Expert



Linux/UNIX/bash



Introduction of Unix & Terminal

Philosophy of Unix

- String small programmes / steps together by **pipes**
- Strung together with **pipes**: **|** or **>**

Pipeline:

Program 1 + file 1 > output 1

Program 1 + file 1 | Program 2 + output 1 | etc.

Introduction of Unix & Terminal

Philosophy of Unix

Advantages:

- Checks each step
- Errors easy to solve
- Computationally efficient
- Flexible
- Updates of programs don't break the whole pipeline

Disadvantages:

- Requires some coding skills
- Good documentation needed for reproducibility

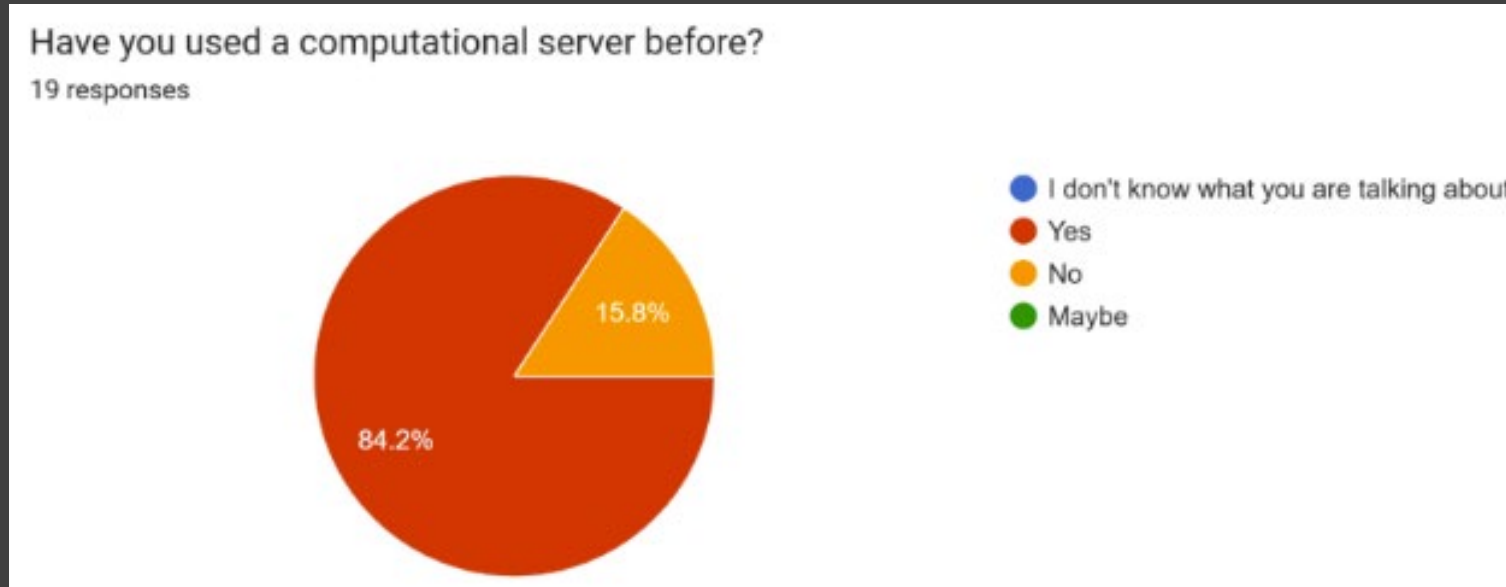
Log-in to LOCAL computer & Explore!

We will switch now between this presentation and GitHub

Exercise 0: Log-in to the computer

Exercise 1: Explore the local computer

Types of server



Cloud server <> physical server

Server / cluster manager / IT

Command line

```
!!! Bitte keine Rechenjobs auf diesem Rechner laufen lassen !!!
```

```
Er dient nur als remote Office-Server.
```

```
RECHENJOBS WERDEN ohne Rückfrage GEKILLED.
```

```
https://www.mi.fu-berlin.de/w/IT/ComputeServer
```

```
Last login: Sun Feb 15 20:55:24 2026 from 87-77-244-95.mna.fu-berlin.de
```

```
k00457149@andorra:~$ ls
```

```
Scan
```

```
k00457149@andorra:~$ pwd
```

```
/home/mi/k00457149
```

```
k00457149@andorra:~$ █
```

Two words on error reporting

„It doesn't work“ 🤔

„I tried program x and it doesn't work“ 😞

„I ran this command with program x and it doesn't work“ 🤔

„I am logged in on the server, loaded the right package, am in the right directory, and this command with program x doesn't work, I already checked if I didn't make any typo's“ 🤔 🤔 🤔

Nodes, CPU's and Memory

Node: a single, self-contained within the server. A server can have multiple nodes.

CPU: the „Central Processing Unit“ – does the actual computing (the more CPU's, the higher computational speed)

RAM: Random Access Memory (RAM); temporary, high-speed storage where the CPU holds data it is working on. „Active memory“

Nodes, CPU's and Memory

Example!

Laptop:

4+ Core (CPU – but same chip)

RAM 16Gb or 32 GB

Mjolnir server:

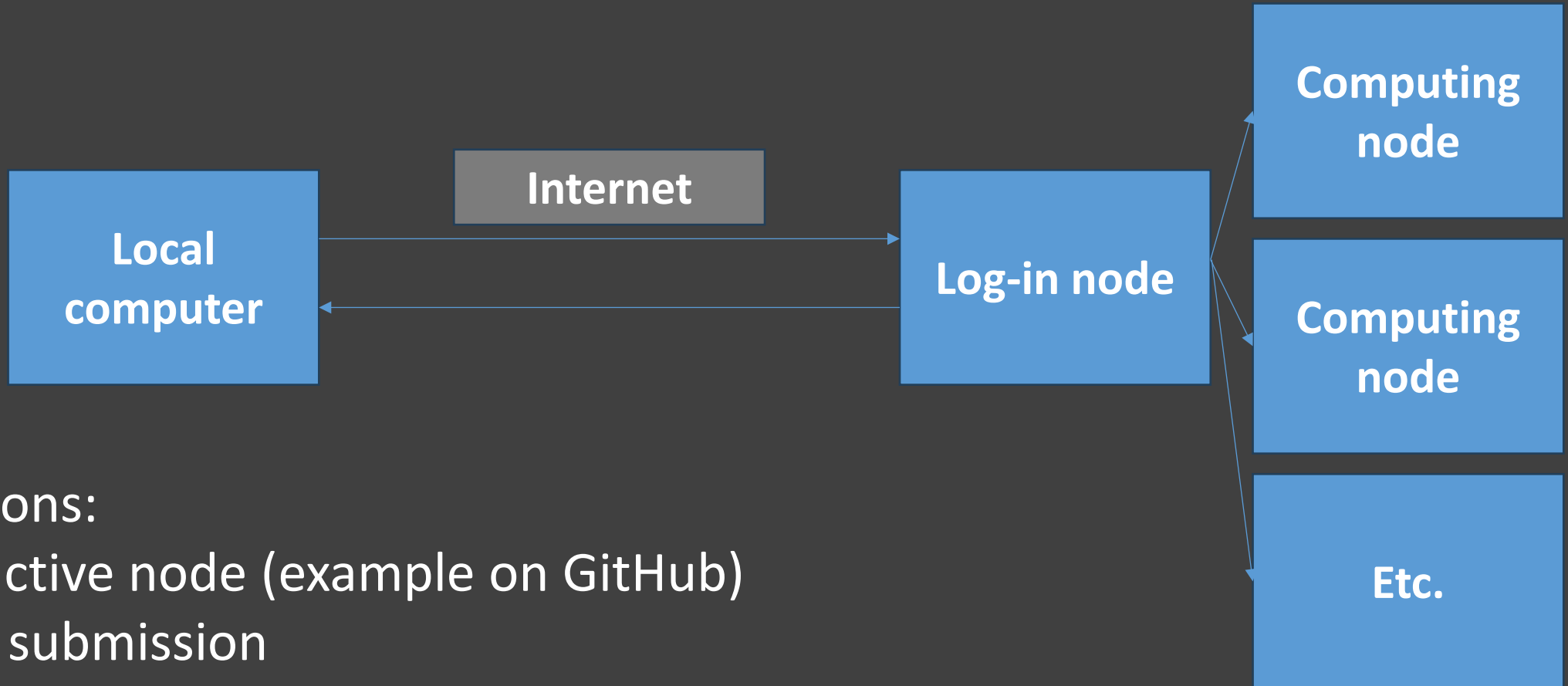
24 Nodes

CPU: 56-256

512-2092 GB RAM

Nodes, CPU's and Memory

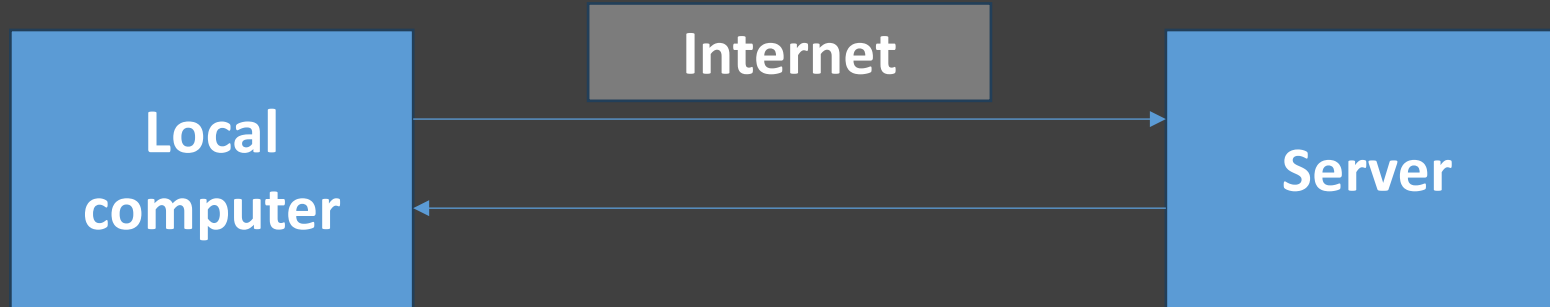
Log-in nodes ... are not **computing** nodes!



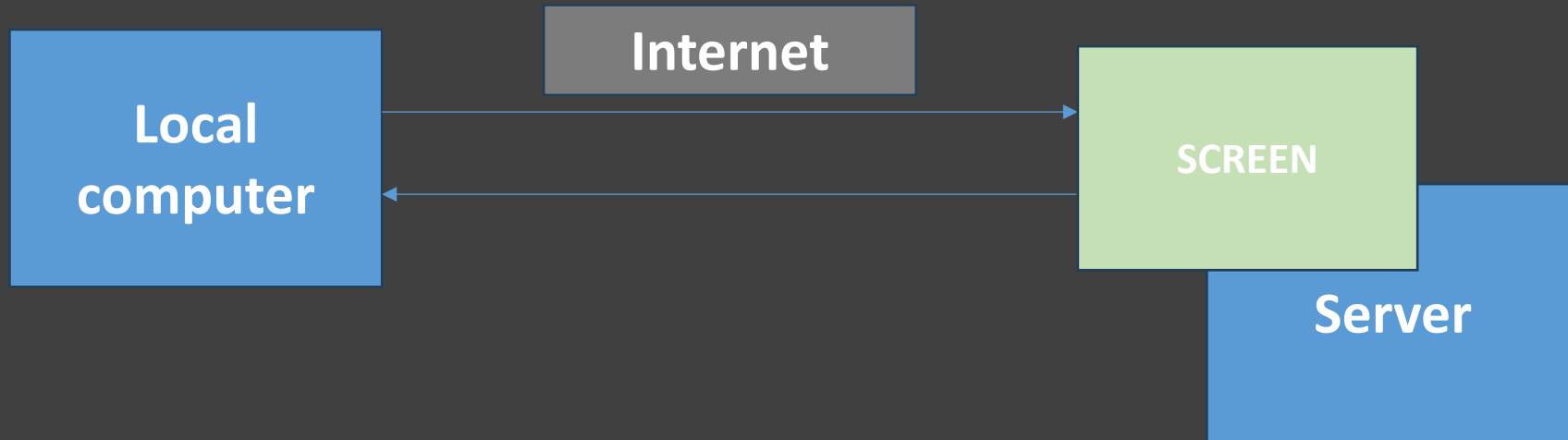
Solutions:
interactive node (example on GitHub)
& job submission

Exercise 2: log in to the server

Local – internet – Server



Local – internet – Screen – Server



Exercise 3: Use a screen

Files & Folders

We can move files between local and server

What is the folder structure?

Expences of data storage

What can we do?

- Zip data
- Remove intermediate files

Files & Folders

```
ssh <username>@<remote_host>
```

```
scp /path/to/local/file <username>@<remote_host>:/path/to/remote/directory
```

Exercise 4: downloading

Wildcards



Using a “*”; it can mean anything e.g.

File_*.fq.gz =

File_1_F.fq.gz

File_1_R.fq.gz

File_2_F.fq.gz

File_2_R.fq.gz

output.txt

File_1_*.fq.gz =

File_1_F.fq.gz

File_1_R.fq.gz

File_2_F.fq.gz

File_2_R.fq.gz

Exercise 5.1 More downloading

Using keys

- Ctrl + C → Cancel a job
- Magic “tab” → Fill out the remaining folder or file name
- “history” → find back what commands you were running before
- “clear” → clear your screen

Exercise 5.2 using the “tab”

Exercise 5.3 & 5.4

Executing commands

```
k00457149@andorra:~$ ls
```

Scan

```
k00457149@andorra:~$ pwd
```

```
/home/mi/k00457149
```

```
k00457149@andorra:~$ █
```

User name @ server: ~\$ COMMAND

Executing commands

```
# run fastqc
```

```
fastqc -o /projects/alberdilab/people/lzh623/Bombina_RNA/fastqc_filtered_results filtered_AHO*.fq.gz
```

Command, parameter (-o), arguments

User manual?



User manual?

```
k00457149@andorra:~$ man screen
```

```
k00457149@andorra:~$ screen --help
```

```
k00457149@andorra:~$ screen -h
```

man screen

SCREEN(1)

General Commands Manual

SCREEN(1)

NAME

`screen` - screen manager with VT100/ANSI terminal emulation

SYNOPSIS

```
screen [ -options ] [ cmd [ args ] ]
screen -r [[pid.]tty[.host]]
screen -r sessionowner/[[pid.]tty[.host]]
```

DESCRIPTION

`Screen` is a full-screen window manager that multiplexes a physical terminal between several processes (typically interactive shells). Each virtual terminal provides the functions of a DEC VT100 terminal and, in addition, several control functions from the ISO 6429 (ECMA 48, ANSI X3.64) and ISO 2022 standards (e.g. insert/delete line and support for multiple character sets). There is a scrollback history buffer for each virtual terminal and a copy-and-paste mechanism that allows moving text regions between windows.

When `screen` is called, it creates a single window with a shell in it (or the specified command) and then gets out of your way so that you can use the program as you normally would. Then, at any time, you can create new (full-screen) windows with other programs in them (including more shells), kill existing windows, view a list of windows, turn output logging on and off, copy-and-paste text between windows, view the scrollback history, switch between windows in whatever manner you wish, etc. All windows run their programs completely independent of each other. Programs continue to run when their window is currently not visible and even when the whole `screen` session is detached from the user's terminal. When a program terminates, `screen` (per default) kills the window that contained it. If this window was in the foreground, the display switches to the previous window; if none are left, `screen` exits. Shells usually distinguish between running as login-shell or sub-shell. `Screen` runs them as sub-shells, unless told otherwise (See shell `.screenrc` command).

Everything you type is sent to the program running in the current window. The only exception to this is the one keystroke that is used to initiate a command to the window manager. By default, each command begins with a control-a (abbreviated C-a from now on), and is followed by one other keystroke. The command character and all the key bindings can be fully customized to be anything you like, though they are always two characters in length.

`Screen` does not understand the prefix C- to mean control, although this notation is used in this manual for readability. Please use the caret notation (^A instead of C-a) as arguments to e.g. the `escape` command or the `-e` option. `Screen` will also print out control characters in caret notation.

The standard way to create a new window is to type C-a c. This creates a new window running a shell and switches to that window immediately, regardless of the state of the process running in the current window. Similarly, you can create a new window with a custom command in it by first binding the command to a keystroke (in your `.screenrc` file or at the C-a : command line) and then using it just like the C-a c command. In addition, new windows can be created by running a command like:

```
screen emacs prog.c
```

from a shell prompt within a previously created window. This will not run another copy of `screen`, but will instead supply the command name and its arguments to the window manager (specified in the `$STY` environment variable) who will use it to create the new window. The above example would start the emacs editor (editing `prog.c`) and switch to its window. - Note that you cannot transport environment variables from the invoking shell to the application (emacs in this case), because it is forked from the parent screen process, not from the invoking shell.

If `/run/utmp` is writable by `screen`, an appropriate record will be written to this file for each window, and removed when the window is terminated. This is useful for working with `talk`, `script`, `shutdown`, `rsend`, `sccs` and other similar programs that use the `utmp` file to determine who you are. As long as `screen` is active on your terminal, the terminal's own record is removed from the `utmp` file. See also C-a L.

GETTING STARTED

Before you begin to use `screen` you'll need to make sure you have correctly selected your terminal type, just as you would for any other termcap/terminfo program. (You can do this by using `test` for example.)

Manual page `screen(1)` line 1 (press h for help or q to quit)

screen -h

```
k00457149@andorra:~$ screen --help
```

```
Use: screen [-opts] [cmd [args]]  
or: screen -r [host.tty]
```

Options:

```
-4          Resolve hostnames only to IPv4 addresses.  
-6          Resolve hostnames only to IPv6 addresses.  
-a          Force all capabilities into each window's termcap.  
-A -[r|R]   Adapt all windows to the new display width & height.  
-c file     Read configuration file instead of '.screenrc'.  
-d (-r)     Detach the elsewhere running screen (and reattach here).  
-dmS name   Start as daemon: Screen session in detached mode.  
-D (-r)     Detach and logout remote (and reattach here).  
-D -RR      Do whatever is needed to get a screen session.  
-e xy       Change command characters.  
-f          Flow control on, -fn = off, -fa = auto.  
-h lines    Set the size of the scrollback history buffer.  
-i          Interrupt output sooner when flow control is on.  
-l          Login mode on (update /var/run/utmp), -ln = off.  
-ls [match] or  
-list       Do nothing, just list our SockDir [on possible matches].  
-L          Turn on output logging.  
-Logfile file Set logfile name.  
-m          ignore $STY variable, do create a new screen session.  
-O          Choose optimal output rather than exact vt100 emulation.  
-p window   Preselect the named window if it exists.  
-q          Quiet startup. Exits with non-zero return code if unsuccessful.  
-Q          Commands will send the response to the stdout of the querying process.  
-r [session] Reattach to a detached screen process.  
-R          Reattach if possible, otherwise start a new session.  
-s shell    Shell to execute rather than $SHELL.
```

Finding which version

```
k00457149@andorra:~$ screen -v  
Screen version 4.09.00 (GNU) 30-Jan-22  
k00457149@andorra:~$ screen --version  
Screen version 4.09.00 (GNU) 30-Jan-22
```

Exercise 6

Coffee break

Most used commands

pwd : Print Working Directory

```
k00457149@andorra:~$ pwd  
/home/mi/k00457149
```

cd : Change Directory

- cd ../
- cd ~
- cd <tab> <tab>

```
k00457149@andorra:~$ ls  
new_file.txt  Scan
```

ls : LiSt

```
k00457149@andorra:~$ cd Scan/  
k00457149@andorra:~/Scan$ ls  
k00457149@andorra:~/Scan$
```

mkdir: MaKe DIRectory

- mkdir Intro_Unix

Most used commands

Bonus; “tree”

```
k00457149@andorra:~/Scan$ tree
```

```
.
```

```
0 directories, 0 files
```

```
k00457149@andorra:~/Scan$ cd ../
```

```
k00457149@andorra:~$ tree
```

```
.
```

```
|— new_file.txt  
|— Scan
```

```
2 directories, 1 file
```


Exercise 7

More most used commands

mv: MoVe <old file> <same file with new name or location>

cp (scp): CoPy <old file> <copy (new) file>

rm: ReMove <a file> (also possible with directories)

- rm * 🤖
- No bin

get = get <something from the web>

Exercise 8.1

cat

cat: conCATenate

cat file 1

cat file1 file2

cat > file1

<write something>

ctrl + D to close file

```
k00457149@andorra:~$ cat > try_cat.txt
bla bla bla
k00457149@andorra:~$ cat try_cat.txt
bla bla bla
k00457149@andorra:~$ rm try_cat.txt
k00457149@andorra:~$
```

Exercise 8.2

Even more “most used” commands

tar ; for tar.gz files! “tarball” archive with multiple files and folders. Tar doesn’t do compression

gzip ; for .gz files! a single file

unzip ; for .zip files! Zip archive can contain multiple files / folders. Zip does compression



Even more “most used” commands

more: view a file and only scroll forwards

less: view a file and scroll back and forth

most: supports multiple files

Exercise 9.1

Even more “most used” commands

head: prints the first 10 lines of a file

tail: prints the last 10 lines of a file

Exercise 9.2 & 9.3

Even more “most used” commands

`clear`: clears your terminal screen



`exit`: leave the connection or screen or interactive node

Searching in files

`grep <pattern> <file>`

pattern can be anything:

“hello”

“GENE_12”

“ACTG”

Good practice to use “”

There are many options; case sensitive, start line, end line ... !



Exercise 10 (and extra)

Searching in files like a PRO

`awk <filters> <file name>`

For tabular files

It recognizes columns (Goodbye Excel!!)

You can indicate which column you want to search in with the “\$”

E.g. \$3 is the third column.

You can also calculate new numbers, e.g. \$3-\$2 !



Exercise 11 (and extra)

Working on zipped files

zcat

zgrep



Piping

Command 1 file 1 > output

Directs the output of command 1 into

Command file 1 | command 2 output

...

```
grep "something" < /path/to/input.file > /path/to/output.file
```

```
$ cat contents.txt | grep file
```

```
0 Aug 9 13:55 file1
```

```
0 Aug 9 13:55 file2
```

```
0 Aug 9 13:55 file3
```

```
0 Aug 9 13:55 file4
```

```
0 Aug 9 13:55 file5
```

Exercise 12

LUNCH BREAK!